

Оценка семантики замененных сущностей

1. Проблематика: что именно требуется оценить

Нам нужно не просто найти сущности и не просто восстановить маски, а оценить влияние конкретной замены на смысл текста. Для каждого упоминания есть исходная сущность e , полный контекст t и кандидат c . Нужно решить, является ли c допустимой заменой в этом контексте.

Positive	the National Air Traffic Controllers Association → NATCA: строки разные, но организация та же; смысл сохраняется.
Negative	the University of Oklahoma → University of Hawaii: тип объекта совпадает, но факт меняется.
Uncertain	Слишком общий <i>alias</i> , неоднозначная аббревиатура или контекст, где даже человек не уверен без внешнего знания. Такие пары не должны попадать в обучение.

Почему эта задача отдельная:

- NER/типизация отвечает “что это за сущность”, но не отвечает “эквивалентна ли эта подстановка”.
- Суммаризационная метрика может не заметить локальную фактическую ошибку, если общий смысл текста похож.
- Строковое сравнение ловит только *identity cases* и пропускает *alias/аббревиатуры*.
- Простые *random negatives* слишком легкие; модель начинает отличать очевидный мусор, а не близкую фактическую ошибку.

Поэтому целевой объект разметки — не сущность сама по себе, а пара *context + original entity + candidate*.

2. Почему “плохие” кандидаты должны быть близкими

Негативный класс не должен быть случайным. Если плохие кандидаты слишком простые, cross-encoder выучит формальные признаки: другой регистр, другой тип слова, явную нерелевантность, странную грамматику. В реальном применении ошибка чаще выглядит правдоподобно.

Тип ошибки	Пример	Чему должна научиться модель
Alias vs original	African Growth and Opportunity Act → AGOA	разные написания могут обозначать один и тот же объект
Близкая организация	National Air Traffic Controllers Association → другая aviation/union organization	похожий домен не означает ту же сущность
Близкий закон	AGOA → другой trade act	юридический текст может быть похожим, но факт меняется
Общий дескриптор	University of Oklahoma → the university	слишком общий кандидат может быть бесполезен как positive
Число	20 percent → 30 percent	локальная величина меняет фактическое утверждение

Практический вывод: negative класс лучше мыслить как changed, а не как bad. Это семантически близкие кандидаты, которые меняют факт или объект.

3. Датасеты, с которыми работаем

В проекте лежат masked-варианты нескольких публичных датасетов. Они тематически неоднородны: news, multi-document news, short news, legal bills, dialogue. Это полезно, потому что подстановка сущностей по-разному ведет себя в новостях, юридических текстах и диалогах.

Dataset / split	Тексты	Маски	Роль в задаче
cnn_dailymail/train	63 777	2 575 378	основной news-корпус; много организаций, персон, географии
multi_news/train	5 190	902 056	мультидокументные новости; длиннее контексты и больше coreference
xsum/train	4 597	99 398	короткие новости; плотный контекст вокруг сущности
billsum/train	3 984	253 956	legal domain; законы, акты, комитеты, номера
samsum/train	77	1 322	dialogue domain; пока малый вклад

Суммарно в slim-инвентаре для текущей proper-name ветки: 80 036 текстов, 19 177 уникальных proper-name сущностей и 230 124 их упоминания. Этого достаточно, чтобы ручная разметка была неадекватна по стоимости, а автоматическая разметка требовала строгого контроля качества.

4. Зачем нужен SQLite-инвентарь

SQLite здесь не просто хранилище результатов. Он фиксирует связь между текстами, масками, сущностями и упоминаниями, чтобы разметка была воспроизводимой и проверяемой.

Что хранится	dataset/split/domain; исходный текст; masked text; число масок; нормализованное написание сущности; все упоминания сущности; короткий и полный контекст; индексы масок.
Что это дает	можно дедуплицировать сущности, видеть все места употребления, выбирать несколько разных контекстов на одну сущность, считать coverage по доменам и типам, делать воспроизводимые срезы для LLM-разметки.
Почему не JSONL-only	для 80k текстов и сотен тысяч упоминаний нужны быстрые выборки: “дай top proper names”, “дай 2 контекста на сущность”, “покажи все пары для этой сущности”, “не бери уже обработанные”.
Как используется в пайплайне	сначала строится инвентарь всех маскированных сущностей и их упоминаний; затем сущности получают типы; далее каждое упоминание маршрутизируется в proper-name / number / common branch.

Аналитически это важно не само по себе как “еще одна база”, а как способ контролировать разметку: видеть, какие домены и типы сущностей покрыты, где повторяются одни и те же названия, какие тексты дают спорные кандидаты и где может возникнуть leakage между train/test.

5. Предварительная типизация сущностей

Перед генерацией кандидатов нужно понять, с каким типом сущности мы работаем. Один и тот же механизм не подходит для NATSA, 20 percent, the bill, John Smith и the university.

Типизация делалась как двухступенчатая процедура. Сначала из всех masked positions извлекается написание сущности и нормализуется для дедупликации. Затем сущности присваиваются coarse group эвристиками: числовые паттерны, capitalization/ proper-name признаки, служебные и доменные термы, common phrases. Для частотных и спорных сущностей дополнительно используется LLM-разметка в фиксированной схеме полей: group, type, confidence, без свободного текста. Итоговый тег хранится вместе с observed tags, доменами, датасетами и количеством упоминаний.

Группа	Сущности	Упоминания	Что означает для разметки
proper_name	776 446	15.85M	полный контекст; alias и hard negatives через LLM
numeric	132 530	3.87M	локальное окно; единицы, даты, проценты, деньги
domain_term	116 109	750k	часто зависит от предметной области и соседних слов
common_entity	17 888	636k	embedding retrieval + фильтрация
junk	23	4.4k	лучше исключать или отправлять в uncertain

Вывод из частотностей: proper names доминируют по разнообразию и числу упоминаний, но числовая ветка тоже велика. Поэтому one-size-fits-all разметка была бы дорогой и шумной.

6. Какие теги присваивали сущностям

Для каждой нормализованной сущности сохранялись два уровня разметки: общий класс group и более конкретный тег type. Это нужно, чтобы выбирать стратегию генерации кандидатов и потом анализировать качество модели по типам.

Тег	Примеры	Как используется
proper_name / ORG	NATCA, University of Oklahoma	полный контекст; alias и близкие changed-кандидаты через LLM
proper_name / PERSON	John Smith, Jessica Runions	нужны проверки на фамилии/частичные имена, чтобы не принять то же лицо за hard negative
proper_name / LAW	AGOA, Clean Air Act	особенно строгая проверка: похожий закон почти всегда меняет факт
proper_name / GPE/LOC/FAC	Oklahoma, the Capitol, Boston	контекст нужен для различения места, объекта и организации
numeric / DATE/MONEY/ PERCENT	2008, \$5 million, 20 percent	локальное окно вокруг числа и ближайшая единица/сущность
common_entity	the company, the bill, the city	embedding retrieval по названию сущности и короткому контексту
domain_term	committee, section, subsection	часто зависит от юридического или новостного домена
UNTYPED/UNKNOWN	частотные, но неоднозначные сущности	не выбрасываются автоматически; требуют контекста или LLM-тега

Важный момент: тег не считается финальной истиной о мире. Он задает маршрут обработки и ограничения для генерации/судейства.

7. Частотности типов после первичной типизации

Ниже — крупнейшие группы group/type в полном инвентаре извлеченных сущностей до отбора узкого proper-name среза. UNTYPED не означает “неизвестно вообще”: это случаи, где coarse group определен, но более тонкий NER-подтип не был надежно восстановлен из исходных масок и контекста.

Группа / тер	Сущности	Упоминания
proper_name / UNTYPED	699 797	9.38M
proper_name / UNKNOWN	58 974	1.05M
proper_name / ORG	11 031	827k
proper_name / GPE	1 568	2.60M
proper_name / PERSON	1 025	643k
proper_name / LAW	906	13k

Группа / тер	Сущности	Упоминания
numeric / DATE	72 757	1.63M
numeric / MONEY	38 973	242k
numeric / CARDINAL	10 789	1.83M
numeric / PERCENT	9 175	136k
domain_term / UNTYPED	111 209	684k
common_entity / UNTYPED	17 532	265k

Аналитический вывод: типы распределены крайне неравномерно. Proper names дают основной объем и требуют полного контекста, но numeric-типы имеют миллионы упоминаний и могут дать дешевый дополнительный supervision при локальной обработке.

8. Рассмотренные варианты решения

Мы обсуждали несколько уровней сложности. Итоговый пайплайн комбинирует их, а не выбирает один универсальный метод.

Подход	Где полезен	Сильная сторона	Проблема
Embedding similarity	common entities, первичный retrieval	дешево строит близких кандидатов	косинусная близость не равна фактической корректности
LLM generator	proper names и hard negatives	может учитывать полный текст и знание мира	стоимость, rate limits, нужен строгий формат вывода
LLM judge	валидация generated pairs	лучше справляется с семантикой контекста	нужен consensus и аудит ошибок и довольно дорог
Cross-encoder	production/inference score	быстро оценивает пару текстов после обучения	качество ограничено качеством разметки

Выбор: LLM используем как offline-разметчик и генератор сложных кандидатов, а итоговый проверяющий модуль делаем как обучаемый cross-encoder.

9. Архитектура разметки

Общая схема:

1	Инвентарь	собрать тексты, маски, сущности, упоминания, контексты, домены
2	Типизация	назначить группе сущности стратегию обработки: proper-name, number, common, ambiguous
3	Генерация	для proper names LLM получает полный текст и просит good aliases + close changed candidates
4	Валидация до судей	убрать identity, совпадающие/частичные названия той же сущности, слишком общие alias, явно слабые negatives
5	LLM judges	несколько независимых судей размечают preserved / changed / uncertain
6	Consensus	positive/negative берутся только при достаточном согласии; uncertain исключается
7	Clean train	удаление duplicates, identity, missing metadata; сохранение text_id/mention_id/domain/type
8	Training	cross-encoder получает два полных текста и учится выдавать score семантической сохранности

Почему это важно: генератор может ошибаться, судья может ошибаться, embedding может ошибаться. Поэтому пайплайн строится как последовательность ограничителей.

10. Proper-name ветка: что делает LLM

Для имен собственных контекст критичен. Одно и то же написание может быть частью названия закона, организации, места или события; аббревиатура может быть неоднозначной. Поэтому LLM получает полный текст вокруг упоминания.

Задача генератора:

- предложить хорошие кандидаты: аббревиатуры, канонические названия, общепринятые aliases;
- предложить hard negatives: другой объект того же типа и близкого домена, который выглядит правдоподобно, но меняет факт;
- не возвращать исходную сущность в другом регистре или с пунктуацией;
- не превращать конкретную сущность в слишком общий дескриптор, если это не сохраняет смысл;
- для каждой пары сохранять тип сущности, mention id, text id, dataset и domain.

Original	Candidate	Judge label	Комментарий
National Air Traffic Controllers Association	NATCA	preserved	аббревиатура той же организации
University of Oklahoma	University of Hawaii	changed	тот же тип объекта, но другой университет
African Growth and Opportunity Act	AGOA	preserved	аббревиатура того же закона

11. Числа и common entities: план обработки

Proper names — самая трудная ветка, но не единственная. Для чисел и common entities нужен другой баланс цены и качества.

Ветка	Как генерировать	Как судить
Numbers	берем окно вокруг конкретного числа: 1-2 токена слева/справа и ближайшую единицу или noun phrase. Гипотеза: числительное обычно относится к сущности перед ним или после него; меняем величину/год/процент/валюту, сохраняя локальную грамматику	часть валидируется правилами; спорные случаи отдавать LLM вместе с коротким окном и найденным anchor
Common entities	embedding retrieval по названию сущности и короткому контексту: очень близкие кандидаты как possible positives, 0.8-0.9 как hard negatives. Гипотеза: обычно энкодеры тренируют как раз быть ближе к синонимам и соответствующим аббревиатурам (как раз обычные существительные), поэтому имеет смысл получать допустимое множество кандидатов через латентное пространство энкодера	LLM/кросс-проверка нужна, потому что близость embedding не гарантирует сохранение факта. Зачастую близкими по cos мере являются антонимы к исходной сущности, если этот антоним образуется приставкой anti, un, non и т.д.
Ambiguous	отдельно рассматривать названия вроде Act, Union, University, которые могут быть частью имени собственного	без контекста лучше маркировать uncertain, чем добавлять шум

Экономия ресурсов достигается маршрутизацией: полный контекст нужен не всем. Для чисел часто хватает короткого окна, для common entities можно предварительно сузить кандидатов embedding-моделью.

12. Контроль качества разметки

Основные источники шума: LLM может вернуть ту же сущность, слишком общий alias, невалидный JSON, или hard negative, который на самом деле является alias. Поэтому качество проверяется до и после судейства.

Контроль	Когда применяется	Что предотвращает
literal equality	до судей и при clean	пары, где кандидат равен исходной сущности после нормализации регистра/пробелов/пунктуации
то же название / та же сущность	после генерации	hard negative вида Jessica Runions → Runions, который может быть той же сущностью
generic alias filter	после генерации	University, Association, Act как слишком общие positives
multi-judge consensus	после LLM judging	случаи, где один судья ошибся или не понял контекст
metadata audit	после clean	пары без dataset/domain/text_id/mention_id, которые нельзя нормально анализировать

В новом full-run все clean rows сохраняют dataset, domain, text_id, mention_id, entity_type, что позволит делать честный анализ качества по доменам и типам.

13. Текущий результат разметки

На первом завершённом окне full-run пайплайн прошёл полный цикл: generation, Mistral judge, repair missing rows, OpenRouter judge, consensus, clean и audit.

Метрика	Значение
total generated pairs	5 164
train pairs after consensus	4 662
clean train rows	4 661
identity rows after clean	0
duplicate rows after clean	0
unique train texts	900
unique train entities	495

Clean rows by type	Rows
ORG	2 839
LAW	1 394
EVENT	372
FAC/GPE/LOC	56

Consensus label	Количество
changed	3 173
preserved	1 489
uncertain	502
clean label 0	3 173
clean label 1	1 488
uncertain in train	0

Clean rows by domain	Rows
legal	2 284
news	2 086
multi_news	170
news_short	121

14. Что показывает качество candidates

Важно проверить отдельно две ветки: good alias generation и hard-negative generation. Первая должна давать preserved, вторая — changed. В реальности alias branch строже: часть “хороших” кандидатов оказывается слишком общей или спорной и уходит в changed/uncertain.

Alias branch	Consensus
preserved	1 483
changed	177
uncertain	488
total	2 148

Hard-negative branch	Consensus
changed	2 996
preserved	6
uncertain	14
total	3 016

Интерпретация:

- hard-negative generation работает надежно: почти все кандидаты действительно меняют смысл;
- alias generation полезен, но требует строгого судейства, потому что LLM иногда предлагает слишком общий replacement;
- identity rows после clean равны нулю, что закрывает проблему “LLM вернула исходную сущность” для обучающего срез.

15. Как обучается проверяющая модель

Cross-encoder получает не только пару строк entity/candidate, а две версии текста: исходную и с подстановкой кандидата. Это позволяет модели учитывать роль сущности в предложении, соседние сущности, домен и фактическое утверждение.

Input	left: полный текст с исходной сущностью; right: тот же текст с кандидатом вместо целевой сущности
Target	1 = preserved, 0 = changed; uncertain исключены
Split	по entity id, чтобы одна и та же сущность не протекала между train/val/test
Base	локальный ModernBERT sequence classifier, дообучаемый на размеченных парах
Почему cross-encoder	он одновременно видит обе версии текста и может оценивать изменение смысла, а не только близость embeddings

Baseline на старом strict clean-срезе: train — 5 947 rows, validation — 759, test — 743. Новый full-run нужен, чтобы увеличить покрытие и сохранить metadata для анализа ошибок.

16. Baseline-результаты обучения

Локальный baseline нужен не как финальная метрика, а как sanity check: разметка обучаемельна, модель начинает отличать preserved от changed, но качество ограничено объемом и шумом раннего среза.

Validation	Value
accuracy, threshold 0.50	0.714
macro F1, threshold 0.50	0.695
best threshold	0.54
macro F1, threshold 0.54	0.702

Test	Value
accuracy, threshold 0.50	0.666
macro F1, threshold 0.50	0.657
positive F1	0.601
negative F1	0.713

По типам видно, что FAC/ORG проще, LAW/EVENT сложнее. Это ожидаемо: законы и события часто требуют большего контекста и внешнего знания. Поэтому следующая оценка должна быть stratified: отдельно по домену, типу сущности и ветке генерации candidates.

17. Выводы

По текущему срезу видно, что задача не сводится к cosine similarity: положительный класс часто представлен alias/аббревиатурами, а отрицательный класс должен быть близким по типу и домену.

Наблюдение	Что это значит	Следствие для модели
Proper names доминируют по частотности и разнообразию	полный контекст и внешнее знание важнее локального совпадения слов	cross-encoder должен видеть полный текст, а не только entity/candidate
Hard negatives почти всегда judged as changed	генератор научился создавать близкие, но фактически неверные кандидаты	это полезнее для обучения, чем random negatives
Alias branch шумнее	часть “хороших” кандидатов слишком общая или спорная	нужен consensus и exclusion of uncertain
LAW/EVENT сложнее ORG/FAC	юридические акты и события требуют больше контекста и точной денотации	метрики нужно смотреть отдельно по типам, а не только overall
Numbers требуют отдельной ветки	число связано с ближайшей сущностью/единицей в локальном окне	дешевле и надежнее размечать их иначе, чем proper names

Главный вывод: разметка должна быть тип-зависимой. Один общий prompt или один embedding threshold даст либо дорогой, либо шумный датасет; качественная разметка на практике требует небольшой мультиагентной системы.